

PAP 2.2 – Python-Auswertung

Wichtige Funktionen für das PAP.

Alle in diesem Dokument geschriebenen Zeilen Code sind von mir eigenständig geschrieben. Für ein sauberes und strukturiertes Dokument wurde eine eigene Pythonlibrary geschrieben, welche insbesondere ans PAP angepasst wurde. Dieses ist unter dem Namen "Papulator" auf meinem GitHub zu finden. Diese soll stetig ausgearbeitet werden. Der Stand dieses Protokolls ist der 29.04.2026.

Inhaltsverzeichnis zur Auswertung

- [Definition der Versuchsvariablen](#)
- [Import aller genutzen Libraries](#)
- [Auswertung der Aufgaben](#)
 - [Aufgabe 1](#)
 - [Aufgabe 2](#)
 - [Aufgabe 3](#)
 - [Aufgabe 4](#)
 - [Aufgabe 5](#)
 - [Aufgabe 6](#)

Definition der Versuchsvariablen

Im Folgenden sind Versuchsvariablen definiert, die für einen besseren Workflow sorgen sollen. Diese werden zum exportieren und überschreiben von Dateien wichtig sein und ermöglichen es, diese Datei für jeden Versuch zu benutzen und lediglich die Variablen zu verändern. Es ist jedoch empfohlen eine Kopie der Forlage für jeden Versuch zu machen, damit dieses Dokument strukturiert bleibt.

Zudem sind wichtige Konstanten definiert, die immer wieder auftauchen.

In [141...

```
versuchsnummer = "253"
versuchsname = "Absorption"
aufgabe = "0z"
plt_save_folder = f"..img/plots/"
```

Import aller genutzen Libraries

In [142...

```
# Eigene PythonLibrary für das Pap
import Papulator as pap
from Papulator import Colors as c
from Papulator import const
from Papulator import Sympy_Symbols as sym

# Numpy für bessere Berechnungen
import numpy as np
from numpy import exp, sqrt, log, pi
from uncertainties import unumpy as unp

# Weiteres für bessere Rechnungen
import pylab as py

import math

# Berechnungen und Plotting
from scipy import odr
import scipy.optimize
from scipy.optimize import curve_fit
from scipy.stats import norm
from scipy.stats import chi2
from scipy.stats import poisson
from scipy.signal import find_peaks
from scipy.signal import argrelextrema, argrelemin, argrelemax
from scipy.special import factorial
from scipy.integrate import quad

import matplotlib.pyplot as plt
import matplotlib.mlab as mlab
import matplotlib.transforms as transforms

import astropy.units as u

# Zum Auslesen von Dateien und ähnlichem
import os
import os.path

import pandas as pd
import csv
import re

from io import StringIO

# Besseres Funktionen handling
import sympy as sp
from sympy import separatevars
```

Auswertung der Aufgaben

Aufgabe 1

```
In [143... U_Betrieb = 520 #V
err_U_Betrieb = 10 #V

r_GMZR = 7 #mm aus durchmesser d = 14mm
```

Aufgabe 2

```
In [144... mess_t_A2 = 300 # s
messungen_A2 = 84

n_0 = messungen_A2 / mess_t_A2
err_n_0 = sqrt(n_0)
```

Aufgabe 3

```
In [145... d_Sr = 6.0

mess_t_Sr = np.concatenate([[30] * 9, [120] * 8])
counts_Sr = np.array([1497, 812, 484, 250, 168, 129, 63, 69, 28, 82, 66, 40, 45, 43, 41, 43, 50])

count_rate_Sr = counts_Sr/mess_t_Sr

err_count_rate_Sr = sqrt(count_rate_Sr)/sqrt(counts_Sr)

# Aluminum Schichten
dicke_Al = np.zeros([len(counts_Sr)])
current_Al_PlattenDicke = 0
for i in range(0, len(counts_Sr)):
    dicke_Al[i] = current_Al_PlattenDicke
    current_Al_PlattenDicke = current_Al_PlattenDicke + 0.3 # mm

n_Beta = 84
n_Beta_0 = n_Beta / 300 # s^-1
err_n_Beta_0 = sqrt(n_Beta_0)/sqrt(300)
```

```
In [146... def expo_decay(d, N_0, mu):
    return N_0 * exp(-mu * (d)) - n_Beta_0
```

```
In [147... plt.figure(figsize=(10,6))

plt.errorbar(
    dicke_Al,
    count_rate_Sr,
    err_count_rate_Sr,
    label = 'Messdaten',
    ls = '',
    capsize = 3.5,
    marker = '.',
)

popt, pcov = curve_fit(expo_decay, dicke_Al, count_rate_Sr, sigma=err_count_rate_Sr, absolute_sigma=True)
counts_reg, mu_reg = popt
err_counts_reg, err_mu_reg = sqrt(np.diag(pcov))

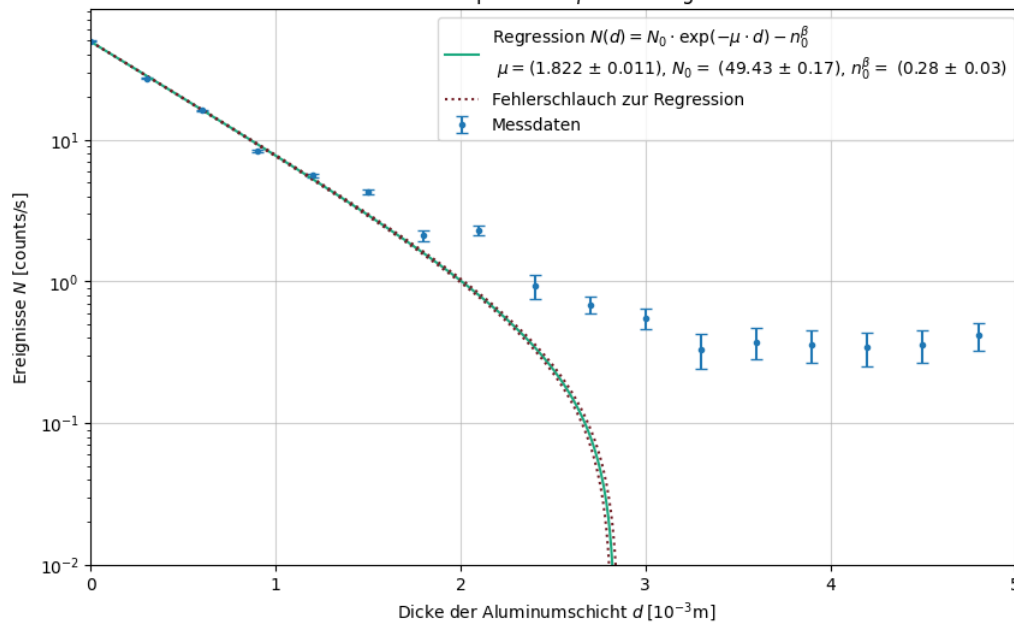
dicke_Al_Space = np.linspace(0, 4.3, 1000)
plt.plot(dicke_Al_Space,
    expo_decay(dicke_Al_Space, counts_reg, mu_reg),
    label = f'Regression  $N(d) = N_0 \cdot \exp(-\mu \cdot d) - n_0 \cdot \beta$   $n \cdot \mu = \text{{pap.round_sig_digs(err_mu_reg, mu_reg)[3]}}$ ,  $N_0 = \text{{pap.round_sig_digs(counts_reg, mu_reg)[3]}}$ ',
    color = c.LIGHT_GREEN
)

plt.plot(dicke_Al_Space,
    expo_decay(dicke_Al_Space, counts_reg, mu_reg - err_mu_reg),
    color = c.WINE_RED,
    ls = ':',
)

plt.plot(dicke_Al_Space,
    expo_decay(dicke_Al_Space, counts_reg, mu_reg + err_mu_reg),
    color = c.WINE_RED,
    ls = ':',
    label = r'Fehlerschlauch zur Regression'
)

plt.yscale('log')
plt.title(r'Absorption von  $\beta$ -Strahlung')
plt.xlabel(r'Dicke der Aluminumschicht  $d$  ;  $\text{{\rm{[10^{-3} m]}}}$ ')
plt.ylabel(r'Ereignisse  $N$  [counts/s]')
plt.legend()
plt.ylim(bottom=1e-2)
plt.xlim(left=0)
plt.grid(True, alpha=0.6)
plt.savefig(f'{plt_save_folder}Absorption_beta_Aluminum.pdf')
plt.show()
```

Absorption von β -Strahlung



```
In [148... max_reichweite = dicke_Al_Space[expo_decay(dicke_Al_Space, counts_reg, mu_reg) <= 1e-14][0]

max_reichweite_left = dicke_Al_Space[expo_decay(dicke_Al_Space, counts_reg, mu_reg-err_mu_reg) <= 1e-14][0]

max_reichweite_right = dicke_Al_Space[expo_decay(dicke_Al_Space, counts_reg, mu_reg+err_mu_reg) <= 1e-14][0]

err_max_reichweite = np.mean([np.abs(max_reichweite - max_reichweite_right), np.abs(max_reichweite - max_reichweite_left)])
print(f'Maximale Reichweite für beta-Strahlung: ({pap.round_sig_digs(err_max_reichweite, max_reichweite)[2]}) mm')
```

Maximale Reichweite für beta-Strahlung: (2.841 $\pm$ 0.017) mm

```
In [149... dichte_Al = 2.7 # g/cm^3
flachen_dichte_Al = 0.13 # g/cm^2
R_beta = dichte_Al * max_reichweite / 10 + flachen_dichte_Al
err_R_beta = dichte_Al * err_max_reichweite / 10

print(f'Die Flächendichte R_beta ist: ({pap.round_sig_digs(err_R_beta, R_beta)[2]}) g/cm^2')
```

Die Flächendichte R_beta ist: (0.897 $\pm$ 0.005) g/cm²

```
In [150... E_lit = 2.274
E_exp = 1.950
err_E_exp = E_exp * 0.1

print(f'Die Standardabweichung beträgt: {pap.std_abw(E_lit, E_exp, err_E_exp)}')
```

Die Standardabweichung beträgt: 1.66

Aufgabe 4

```
In [151... counts_Co = np.array([631, 479, 313, 263, 201, 152, 112, 108, 73, 59, 59])
t_mess_A4 = np.array([60] * len(counts_Co))
```

```
count_rate_Co = counts_Co / t_mess_A4
err_count_rate_Co = sqrt(count_rate_Co)/sqrt(counts_Co)
```

```
dicke_Pb = np.zeros([len(counts_Co)])
current_Pl_Dicke = 0
for i in range(0, len(counts_Co)):
    dicke_Pb[i] = current_Pl_Dicke
    current_Pl_Dicke = current_Pl_Dicke + 0.5 #cm
```

```
In [152... plt.figure(figsize=(10,6))
```

```
plt.errorbar(
    dicke_Pb,
    count_rate_Co,
    err_count_rate_Co,
    label = 'Messdaten',
    ls = '',
    capsize = 3.5,
    marker = '.',
)
```

```
n_Beta_0 = 0.12
err_n_Beta_0 = sqrt(n_Beta_0)/sqrt(60)
```

```
def expo_decay(d, N_0, mu):
    return N_0 * exp(-mu * d) - n_Beta_0
```

```
popt, pcov = curve_fit(expo_decay, dicke_Pb, count_rate_Co, sigma=err_count_rate_Co, absolute_sigma=True)
counts_reg_gamma, mu_reg_gamma = popt
err_counts_reg_gamma, err_mu_reg_gamma = sqrt(np.diag(pcov))
```

```
dicke_Pb_Space = np.linspace(0, 5.5, 1000)
plt.plot(dicke_Pb_Space,
```

```
    expo_decay(dicke_Pb_Space, counts_reg_gamma, mu_reg_gamma),
    label = f'Regression  $N(d) = N_0 \cdot \exp(-\mu \cdot d) - n_0^{\gamma}$ ',
    color = c.LIGHT_GREEN
)
```

```
plt.plot(dicke_Pb_Space,
    expo_decay(dicke_Pb_Space, counts_reg_gamma, mu_reg_gamma - err_mu_reg_gamma),
    color = c.WINE_RED,
```

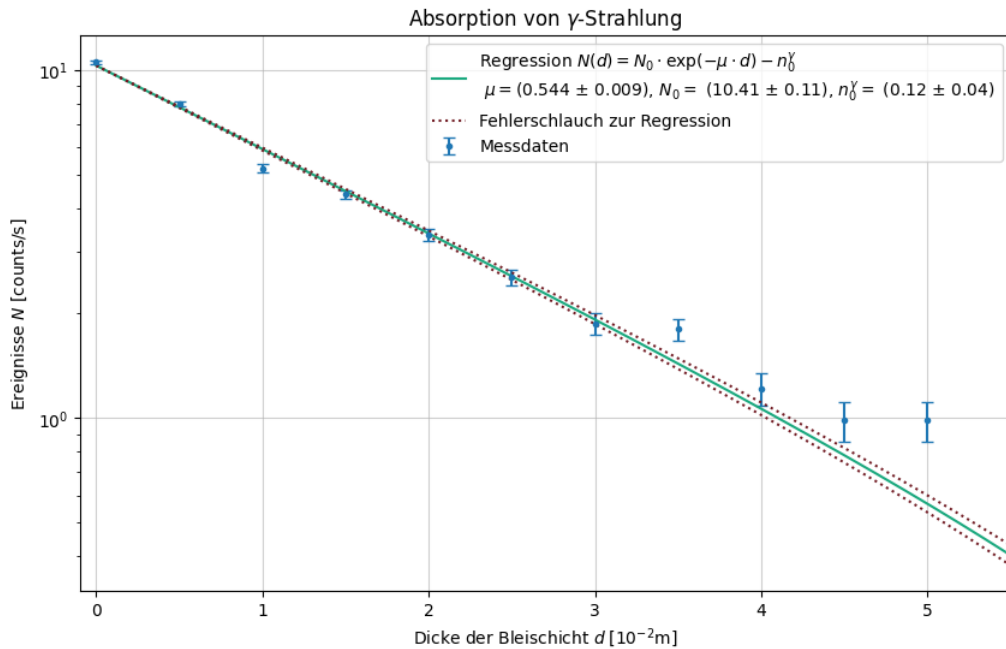
```

ls = ':'
)

plt.plot(dicke_Pb_Space,
         expo_decay(dicke_Pb_Space, counts_reg_gamma, mu_reg_gamma + err_mu_reg_gamma),
         color = c.WINE_RED,
         ls = ':',
         label = r'Fehlerschlauch zur Regression'
        )

plt.yscale('log')
plt.title(r'Absorption von $\gamma$-Strahlung')
plt.xlabel(r'Dicke der Bleischicht $d$; $\mathrm{[10^{-2} \text{ m}]}$')
plt.ylabel(r'Ereignisse $N$ [counts/s]')
plt.legend()
# plt.ylim(bottom=1e-2)
plt.xlim(left=-0.1, right=5.5)
plt.grid(True, alpha=0.6)
plt.savefig(f'{plt_save_folder}Absorption_gamma_Blei.pdf')
plt.show()

```



```

In [153... # Massenschwähungskoeffizient
dichte_Pb = 11.34 # g/cm⁻³

mu_rho_Pb = mu_reg_gamma / dichte_Pb
err_mu_rho_Pb = err_mu_reg_gamma / dichte_Pb

expo_num = 2
expo = 10 ** expo_num
print(f'Der Massenschwähungskoeffizient für Blei ist: ({pap.round_sig_digs(err_mu_rho_Pb * expo, mu_rho_Pb * expo)[2]}) * 10^{-(expo_num)} cm²/g')
Der Massenschwähungskoeffizient für Blei ist: (4.79 ± 0.08) * 10⁻² cm²/g

```

```

In [154... E_gamma_exp = 1.55
err_E_gamma_exp = E_gamma_exp * 0.1
E_gamma_lit_1 = 1.173
E_gamma_lit_2 = 1.333

print(pap.std_abw(E_gamma_exp, E_gamma_lit_1, err_E_gamma_exp))
print(pap.std_abw(E_gamma_exp, E_gamma_lit_2, err_E_gamma_exp))
print(pap.std_abw(E_gamma_exp, (E_gamma_lit_2 + E_gamma_lit_1)/2, err_E_gamma_exp))

2.43
1.4
1.92

```

Aufgabe 5

```

In [155... counts_Co_Zyl = np.array([8444, 2473, 618])
err_counts_Co_Zyl = sqrt(counts_Co_Zyl)
t_mess_A5 = np.array([60] * len(counts_Co_Zyl))
abst_A5 = np.array([5, 10, 20]) # cm
err_abst_A5 = 1.0 # cm

nA = counts_Co_Zyl/t_mess_A5 - n_0 # s⁻¹
err_nA = np.sqrt((err_counts_Co_Zyl/t_mess_A5)**2 + err_n_0**2)

print('Counts')
for i in range(3):
    print(pap.round_sig_digs(err_counts_Co_Zyl[i], counts_Co_Zyl[i])[2])

print('-' * 15)

print('Counts pro Sekunde')
for i in range(3):
    print(pap.round_sig_digs(err_nA[i], nA[i])[2])

r_ZR = 0.7 # cm
l_ZR = 4 # cm
eps = 0.04 # Siehe Praktikumschrift

```

```
Counts
8440 \pm 90
2470 \pm 50
618 \pm 25
-----
Counts pro Sekunde
140.5 \pm 1.6
40.9 \pm 1.0
10.0 \pm 0.7
```

```
In [156... #Ohne Korrekturen
func_aktivitat = (4 * sym.n* sym.d**2)/(sym.epsilon * sym.r**2) * (1/2)

values = np.column_stack([
    nA ,err_nA,
    abst_A5, [err_abst_A5] * len(abst_A5),
    [eps] * len(abst_A5),
    [r_ZR] * len(abst_A5)
])

temp = pap.do_it(func_aktivitat, [sym.n, sym.d, sym.epsilon, sym.r], values, [sym.epsilon, sym.r], False)

aktivitat = temp[:,0]
err_aktivitat = temp[:,1]

expo_num = -5
expo = 10 ** expo_num
for i, n in enumerate(aktivitat):
    print(f'({pap.round_sig_digs(err_aktivitat[i] * expo, aktivitat[i] * expo)[2]}) * 10^{{-expo_num}}')

(3.6 \pm 1.4) * 10^5
(4.2 \pm 0.8) * 10^5
(4.1 \pm 0.5) * 10^5
```

```
In [157... # Mit Korrektur Korrekturen
func_aktivitat = (4 * sym.n* (sym.d + sym.l/2)**2)/(sym.epsilon * sym.r**2) * (1/2)

values = np.column_stack([
    nA ,err_nA,
    abst_A5, [err_abst_A5] * len(abst_A5),
    [l_ZR] * len(abst_A5),
    [eps] * len(abst_A5),
    [r_ZR] * len(abst_A5)
])

temp = pap.do_it(func_aktivitat, [sym.n, sym.d, sym.l, sym.epsilon, sym.r], values, [sym.l, sym.epsilon, sym.r], False)

aktivitat_korr = temp[:,0]
err_aktivitat_korr = temp[:,1]

expo_num = -5
expo = 10 ** expo_num
for i, n in enumerate(aktivitat):
    print(f'({pap.round_sig_digs(err_aktivitat_korr[i] * expo, aktivitat_korr[i] * expo)[2]}) * 10^{{-expo_num}}')

(7.0 \pm 2.0) * 10^5
(6.0 \pm 1.0) * 10^5
(4.9 \pm 0.6) * 10^5
```

```
In [173... # Korrekturfaktor
korr1 = aktivitat_korr/aktivitat
err_korr1 = korr1* np.sqrt((err_aktivitat_korr/aktivitat_korr)**2 + (err_aktivitat/aktivitat)**2)

print('Korrekturfaktoren')
for i, n in enumerate(korr1):
    print(f'({pap.round_sig_digs(err_korr1[i], korr1[i])[2]})')

Korrekturfaktoren
2.0 \pm 1.0
1.4 \pm 0.4
1.21 \pm 0.20
```

```
In [159... #Dichte der Präperatkapsel
rho_kaps = 7.9 # g/cm^3

#Dicke der Präperatkapsel
dick_kaps = 0.14 # cm
```

```
In [196... func_aktivitat_3 = sp.exp(sym.mu * sym.rho * sym.b) * (4 * sym.n * (sym.d + sym.l/2)**2)/(sym.epsilon * sym.r**2) * (1/2)

values = np.column_stack([
    [mu_rho_Pb] * len(abst_A5), [err_mu_rho_Pb] * len(abst_A5),
    nA ,err_nA,
    abst_A5, [err_abst_A5] * len(abst_A5),
    [l_ZR] * len(abst_A5),
    [eps] * len(abst_A5),
    [r_ZR] * len(abst_A5),
    [rho_kaps] * len(abst_A5),
    [dick_kaps] * len(abst_A5)
])

temp = pap.do_it(func_aktivitat_3, [sym.mu, sym.n, sym.d, sym.l, sym.epsilon, sym.r, sym.rho, sym.b], values, [sym.l, sym.epsilon, sym.r, sym.rho, sym.b], False)

aktivitat_korr2 = temp[:,0]
err_aktivitat_korr2 = temp[:,1]

expo_num = -5
expo = 10 ** expo_num
for i, n in enumerate(aktivitat):
    print(f'({pap.round_sig_digs(err_aktivitat_korr2[i] * expo, aktivitat_korr2[i] * expo)[2]}) * 10^{{-expo_num}}')

(7.4 \pm 2.1) * 10^5
(6.3 \pm 1.1) * 10^5
(5.2 \pm 0.6) * 10^5
```

```
In [192... # Präparateigenschaften
lit_Aktivitat_SN_372 = 445000 # Bq
lit_Nennaktivitat = 3700000 # Bq

#Halbwärtszeit von Co60
```

```
t_halb = 5.27 * 365.25 # d
#Tage zwischen Herstellung und Versuch
T = 5950
```

```
A_theo = 3700000 * 2**(-T/t_halb)
print(f'Theoretische Aktivität: A = {A_theo:.0f}')
```

Theoretische Aktivität: A = 434192

```
# Abweichungen
print(f'Ohne Korrektur: {pap.std_abw(A_theo, aktivitat, err_aktivitat)}')
print(f'Erste Korrektur: {pap.std_abw(A_theo, aktivitat_korr, err_aktivitat_korr)}')
print(f'Zweite Korrektur: {pap.std_abw(A_theo, aktivitat_korr2, err_aktivitat_korr2)}')
```

Ohne Korrektur: [0.53 0.2 0.51]
Erste Korrektur: [1.33 1.65 1.09]
Zweite Korrektur: [1.45 1.87 1.49]

Aufgabe 6

```
druck = np.array([20, 124, 237, 325, 435, 524, 650, 841, 1002])
counts_durck = np.array([5224, 5104, 5017, 4228, 87, 65, 71, 76, 61])
```

```
# Wichtiger Druck bereich 320 bis 420
wichtig_druck = np.array([315, 343, 363, 387, 403, 425])
counts_wichtig_druck = np.array([4545, 3437, 2233, 847, 340, 102])
```

```
p_unsorted = np.concatenate([druck, wichtig_druck])
n_unsorted = np.concatenate([counts_durck, counts_wichtig_druck])
```

```
p = np.sort(p_unsorted, descending = False)
n = np.sort(n_unsorted, descending = True)/60
err_n = sqrt(n)/sqrt(60)
data = np.column_stack([p_unsorted,n_unsorted])
data = np.sort(data, axis = 1, descending = True)
data = np.column_stack(data)
```

```
# p = data[0]
# n = data[1]/60
```

```
def linear(m, x, b):
    return m * x + b
```

```
plt.figure(figsize=(12,6))
```

```
left, right = 0, len(p)
```

```
plt.errorbar(
    p[left:right],
    n[left:right],
    err_n[left:right],
    label = 'Messdaten',
    ls = '',
    capsize = 3.5,
    marker = '.',
)
```

```
N_max = n[0]
```

```
plt.hlines([n[0], (n[0] - n[-1])/2], 0, p[right - 1],
          label=f'Maximale Zählrate:  $N_{\text{max}}$  = ({pap.round_sig_digs(err_n[0], n[0])[2]} \; \; \; \mathrm{s^{-1}}) \; \; \; \text{nHalbe Zählrate: } N_{\text{1/2}} = ({pap.r
```

```
left_reg, right_reg = 3, 9
popt, pcov = curve_fit(linear, p[left_reg:right_reg], n[left_reg:right_reg], sigma= err_n[left_reg:right_reg],absolute_sigma=True)
```

```
steig, absch = popt
err_steig, err_absch = np.diag(pcov)
x = np.linspace(300,425,100)
```

```
plt.plot(
    x,
    linear(x, *popt),
    label = f'Linear Regression  $N(p) = m \cdot p + b$   $n_{m=}$ {pap.round_sig_digs(err_steig, steig)[2]}, \; \; \; b={pap.round_sig_digs(err_absch, absch)[2]}$'
)
```

```
plt.errorbar(
    p[left_reg:right_reg],
    n[left_reg:right_reg],
    err_n[left_reg:right_reg],
    label = 'Regressionsdaten',
    ls = '',
    capsize = 3.5,
    marker = '.',
)
```

```
# plt.plot(
#     p[left:right],
#     n[left:right],
#     label = 'Interpolation',
#     ls=':',
#     color = c.BLUE
# )
```

```
plt.title(r'Absorption von  $\alpha$ -Strahlung')
plt.xlabel(r'Luftdruck  $p$ ;  $\mathrm{[hPa]}$ $')
plt.ylabel(r'Ereignisse  $N$  [counts/s]')
plt.legend()
plt.grid(True, alpha=0.6)
plt.xlim(0, 1100)
plt.ylim(0, 90)
```

```
# Small Window
a = plt.axes([.55, .25, .325, .325], facecolor = 'white')
```

```

# Messurment data
plt.errorbar(
    p[left:right],
    n[left:right],
    err_n[left:right],
    label='Messdaten',
    marker = '.',
    ls = '-',
    capsize=3.5
)

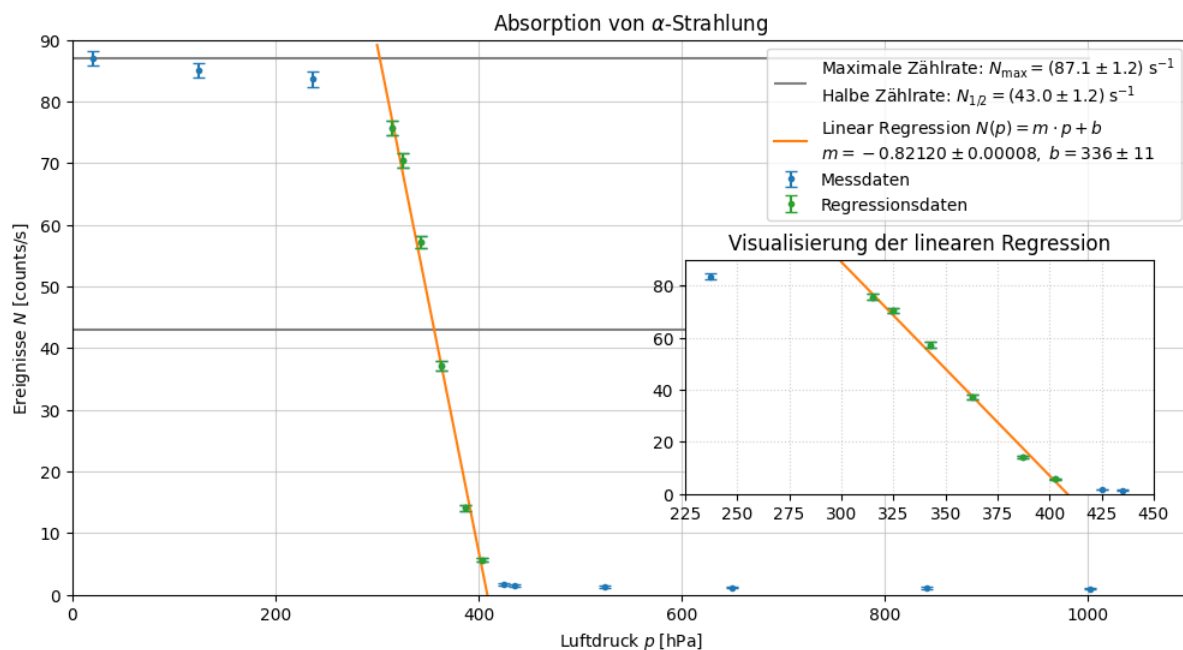
plt.plot(
    x,
    linear(x, *popt),
)

plt.errorbar(
    p[left_reg:right_reg],
    n[left_reg:right_reg],
    err_n[left_reg:right_reg],
    ls = '-',
    capsize = 3.5,
    marker = '.',
)

plt.xlim(225, 450)
plt.ylim(0, 90)
plt.title('Visualisierung der linearen Regression')
plt.grid(alpha = 0.6, ls=':')

plt.savefig(f'{plt_save_folder}Absorption_alpha_Luft.pdf')
plt.show()

```



In [167...

```

sym_N_max = sp.symbols(r'N_{\text{{max}}}')
func_p_half = (sym_N_max - sym.b)/sym.m

values = np.array([
    n[0], err_n[0],
    absch, err_absch,
    steig, err_steig
])

temp = pap.do_it(func_p_half, [sym_N_max, sym.b, sym.m], values)

p_half = temp[0][0]
err_p_half = temp[0][1]

print(f'Der Druck für N_1/2 beträgt: ({pap.round_sig_digs(err_p_half, p_half)[2]}) hPa')

```

gegebene Funktion:

$$\frac{1}{m}(N_{\{\max\}} - b)$$

LaTeX kopieren

Formel des absoluten Fehlers:

$$\sqrt{\frac{1}{m^4} \left(m^2 \left(\Delta N_{\{\max\}}^2 + \Delta b^2 \right) + \Delta m^2 (N_{\{\max\}} - b)^2 \right)}$$

LaTeX kopieren

Formel des relativen Fehlers:

$$\sqrt{\frac{\Delta N_{\{\max\}}^2}{(N_{\{\max\}} - b)^2} + \frac{\Delta b^2}{(N_{\{\max\}} - b)^2} + \frac{\Delta m^2}{m^2}}$$

LaTeX kopieren

Der Druck für N_1/2 beträgt: (303 ± 14) hPa

In [168...

```

sym_p0, sym_s0 = sp.symbols(r'p_0, s_0')

```

```
func_reichweite_alpha = sym.p/sym_p0 * sym_s0

values = np.array([
    p_half, err_p_half,
    1013,
    4.45, 0.05
])

temp = pap.do_it(func_reichweite_alpha, [sym.p, sym_p0, sym_s0], values, [sym_p0])

s_1 = temp[0][0]
err_s_1 = temp[0][1]

print(f'Die Reichweite s_1 betragt: s_1 = ({pap.round_sig_digs(err_s_1, s_1)[2]}) cm')
```

gegebene Funktion:

$$\frac{ps_0}{p_0}$$

LaTeX kopieren

Formel des absoluten Fehlers:

$$\sqrt{\frac{1}{p_0^2}(p^2\Delta s_0^2 + s_0^2\Delta p^2)}$$

LaTeX kopieren

Formel des relativen Fehlers:

$$\sqrt{\frac{\Delta s_0^2}{s_0^2} + \frac{\Delta p^2}{p^2}}$$

LaTeX kopieren

Die Reichweite s_1 betragt: s_1 = (1.33 \pm 0.06) cm

In [169...

```
rho_g = 2.35 #mg/cm^2

s_2 = rho_g / 1.43 * 1
print(f's_2 = ({s_2:.2f}) cm')

s_3 = 0.68

s = s_1 + s_2 + s_3

print(f'Die Reichweite s betragt: s = ({pap.round_sig_digs(err_s_1, s)[2]}) cm')
```

s_2 = (1.64) cm

Die Reichweite s betragt: s = (3.65 \pm 0.06) cm

In [170...

```
E_alpha = 5.25
err_E_alpha = E_alpha * 0.1

print(fr'E_Alpha: ({pap.round_sig_digs(err_E_alpha, E_alpha)[2]}) MeV')

lit_E_alpha = 5.48

print(fr'Standardabweichung: {pap.std_abw(E_alpha, lit_E_alpha, err_E_alpha)}\sigma')
```

E_Alpha: (5.2 \pm 0.5) MeV

Standardabweichung: 0.44\sigma